

Proyecto 1 – Mineral Global Tracker

Curso: Taller de Programación

Grupo: 01

Estudiante: Fabián Gerardo Cambronero Núñez

Carné: 2026079420

Profesor: Diego Mora Rojas

Fecha de entrega: 14/05/26

Periodo: 1 Semestre 2026

Índice

Introducción	3
Descripción del problema	3
Análisis de resultados	4
Funciones desarrolladas correctamente:	4
Problemas encontrados	5
Visualización del mapa	5
Organización de la pantalla de mantenimiento	5
Manejo de datos en archivos	5
Uso de filas y columnas en Tkinter	5
Dificultad.....	6
Conclusiones	6
Estadística de tiempos	7
Tabla:	7

Introducción

El proyecto corresponde al desarrollo del juego Mineral Global Tracker, juego que fue elaborado en Python que combina herramientas de mantenimiento geográfico con una dinámica interactiva de exploración y captura de minerales. La interfaz gráfica fue realizada utilizando la biblioteca Tkinter, permitiendo a los usuarios interactuar mediante interfaces visuales intuitivas tanto para la administración de datos como para jugar a explorar el mundo buscando minerales.

Durante el desarrollo del proyecto se aplicaron distintos conceptos fundamentales de programación, entre ellos el uso de funciones, listas anidadas, manejo de archivos, validaciones, estructuras de control, interfaces visuales y organización lógica de la información. Se trabajó en una interfaz diseñada para facilitar la interacción del usuario en la utilización de las diferentes funciones que ofrece Mineral Global Tracker.

Descripción del problema

El proyecto Mineral Global Tracker surge a partir de la necesidad de desarrollar un sistema capaz de administrar información geográfica relacionada con minerales, al mismo tiempo ofrecer una experiencia interactiva para los usuarios mediante un entorno visual. El problema principal consistía en crear una aplicación que permitiera organizar datos geográficos de manera jerárquica, realizar consultas y análisis sobre dichos datos y gestionar la interacción de jugadores dentro de un sistema de exploración de minerales.

La información debe organizarse utilizando estructuras de datos anidadas, donde cada país contiene estados, cada estado contiene condados y cada condado almacena coordenadas geográficas junto con una lista de minerales asociados. A partir de esta estructura, el sistema debía permitir diferentes operaciones de mantenimiento y consulta, tales como listar países, estados y condados, agregar o eliminar minerales, registrar nuevos condados y localizar regiones mediante coordenadas geográficas.

Otro de los problemas planteados fue el manejo de coordenadas geográficas en formato de grados, minutos y segundos, incluyendo su conversión a formato decimal para facilitar cálculos y validaciones. Esto permitió implementar funcionalidades adicionales como la búsqueda de condados por coordenadas, el cálculo aproximado del área de un condado y la detección de traslapes entre regiones geográficas.

Además de la administración de datos, el proyecto debía incorporar un componente interactivo orientado al usuario. Para ello se desarrolló un modo de juego denominado Mineral Hunter, en el cual los jugadores exploran coordenadas dentro del mapa para descubrir y capturar minerales disponibles en diferentes condados. El sistema debía registrar automáticamente las capturas realizadas, controlar los intentos disponibles de cada partida y almacenar el historial de progreso de los jugadores.

El desarrollo también requería la implementación de interfaces gráficas utilizando la biblioteca Tkinter, permitiendo que todas las operaciones pudieran ejecutarse de forma visual e intuitiva. Asimismo, la información del sistema debía almacenarse de manera persistente mediante archivos de texto, de forma que los datos del mapa y de los jugadores permanecieran guardados entre ejecuciones del programa.

Análisis de resultados

Funciones desarrolladas correctamente:

Durante las pruebas podemos concluir que las siguientes funciones se implementaron de manera correcta y cumplen con el funcionamiento solicitado en los requerimientos de nuestro programa.

- ✓ Listado de países, estados y condados almacenados en el mapa.
- ✓ Consulta de minerales asociados a cada condado.
- ✓ Agregado y eliminación de minerales dentro de los condados.
- ✓ Registro y eliminación de condados
- ✓ Conversión de coordenadas geográficas a formato decimal.
- ✓ Búsqueda de condados mediante coordenadas ingresadas por el usuario.
- ✓ Cálculo aproximado del área de los condados.
- ✓ Detección de traslapes entre regiones geográficas.
- ✓ Análisis de minerales por país.
- ✓ Identificación del condado con mayor cantidad de minerales.
- ✓ Registro de jugadores y almacenamiento de partidas.
- ✓ Captura de minerales dentro del modo de juego.
- ✓ Visualización gráfica del mapa mediante Tkinter.
- ✓ Persistencia de datos utilizando archivos de texto.

Además el sistema logra integrar correctamente la parte administrativa y la parte interactiva del juego, permitiendo que el usuario pueda navegar entre ambas sin perder la información almacenada.

Funciones o aspectos que presentan limitaciones:

Aunque el sistema cumple con la mayoría de los requerimientos, durante las pruebas también se identificaron algunos aspectos que podrían mejorarse o que presentan ciertas limitaciones

- Se utilizan múltiples variables globales, lo cual puede generar problemas de control y organización en proyectos más grandes.
- Algunas validaciones de entrada son limitadas, especialmente en coordenadas y formatos ingresados por el usuario.
- El almacenamiento de información mediante eval() representa un riesgo de seguridad y no es una práctica recomendada.

- La interfaz gráfica puede presentar desorden visual cuando se ingresan grandes cantidades de información.
- El sistema no incluye manejo avanzado de errores ni protección ante modificaciones incorrectas en los archivos de texto.

Problemas encontrados

Visualización del mapa

Uno de los primeros problemas encontrados fue la representación gráfica del mapa dentro de la interfaz. Al inicio, el mapa se mostraba de forma extraña, ya que los condados no se veían bien distribuidos y los puntos ingresados por el jugador no siempre eran fáciles de identificar. Para corregir esto fue necesario ajustar la forma en que las coordenadas se convertían a posiciones dentro del Canvas, tomando en cuenta los límites del mapa, el tamaño disponible y los márgenes visuales.

Organización de la pantalla de mantenimiento

Otro problema importante se presentó en la sección de mantenimiento. Esta parte del sistema contiene muchas funciones, como listar datos, agregar minerales, eliminar minerales, agregar condados, revisar coordenadas y consultar análisis del mapa. Debido a esto, fue necesario organizar cuidadosamente los botones, entradas de texto y paneles de resultado para que la interfaz fuera entendible y fácil de usar. Se trabajó en separar las opciones por secciones para evitar que la pantalla se viera saturada.

Manejo de datos en archivos

También surgieron dificultades al guardar y cargar la información desde archivos de texto. Inicialmente se presentaron problemas porque se estaba utilizando una codificación distinta a UTF-8, lo que causaba errores al leer ciertos caracteres y afectaba el manejo de datos separados por comas. Esto fue especialmente importante porque el sistema utiliza coordenadas y listas de minerales que dependen de una lectura correcta del archivo. Para solucionarlo, se configuró la lectura y escritura de archivos utilizando codificación UTF-8.

Uso de filas y columnas en Tkinter

Otra dificultad fue el uso del sistema de filas y columnas de Tkinter mediante `grid()`. Al ser una herramienta nueva durante el desarrollo, al principio fue complicado alinear correctamente etiquetas, entradas y botones dentro de cada pantalla. Sin embargo, con pruebas y ajustes se logró comprender mejor su funcionamiento, lo que permitió construir interfaces más ordenadas y visualmente estables.

Integración entre lógica e interfaz

Finalmente, también fue necesario ajustar la comunicación entre las funciones internas del programa y la interfaz gráfica. Algunas acciones, como buscar

coordenadas, capturar minerales o mostrar resultados, dependían de que los datos se actualizarán correctamente en pantalla. Esto requirió revisar el flujo del programa para que la información mostrada al usuario coincidiera con los datos almacenados en el sistema.

Dificultad

El proyecto presenta una dificultad media debido a que integra diferentes áreas de programación dentro de una sola aplicación funcional. Durante el desarrollo fue necesario trabajar con estructuras de datos anidadas, manejo de archivos, cálculos matemáticos, validación de información y diseño de interfaces gráficas utilizando Tkinter.

Además, el sistema no solo administra información geográfica, sino que también incorpora un modo de juego interactivo, lo que aumentó la complejidad de la lógica y la comunicación entre las distintas partes del programa. La organización visual de las ventanas, el manejo de coordenadas y la persistencia de datos representaron retos importantes que requirieron investigación, pruebas y correcciones constantes.

Aunque el proyecto no utiliza herramientas avanzadas de bases de datos o librerías externas complejas, sí demandó una buena planificación y comprensión de conceptos fundamentales de programación para lograr un sistema estable, organizado y funcional.

Conclusiones

El desarrollo del proyecto Mineral Global Tracker fue una oportunidad real para poner en práctica muchos de los conceptos que habíamos ido aprendiendo a lo largo del curso. No se trató solo de escribir código, sino hacer que distintas piezas como estructuras de datos, manejo de archivos, validaciones, cálculos matemáticos e interfaces gráficas funcionaran juntas como un sistema coherente y usable.

En el camino también aprendimos cosas que no estaban en el programa. El diseño de interfaces gráficas, por ejemplo, nos exigió pensar de manera distinta: organizar filas, columnas y elementos visuales requiere una lógica propia. También entendimos, lo importante que es mantener una estructura de datos limpia y elegir bien las codificaciones al trabajar con archivos.

Los problemas que surgieron, desde representar visualmente el mapa hasta leer correctamente los datos o acomodar la interfaz como queríamos, no fueron obstáculos sino parte del aprendizaje. Cada error que encontramos nos obligó a detenernos, analizar y buscar una solución mejor.

Estadística de tiempos

Tabla:

Análisis de requerimientos	1:30 hora
Diseño de la aplicación	2 horas
Investigación de funciones	2 horas
Programación	10 horas
Pruebas	2 horas
Elaboración documento	4 horas